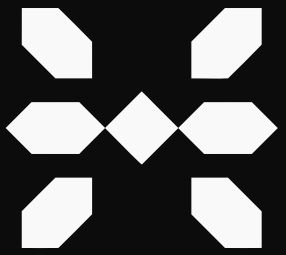
HOOX · AXIS · DOCUMENTATION



PLATFORM ENGINEERS · DEPLOYERS

DEVOPS MANUAL

Local dev, build/serve, Cloudflare, VPS, CORS, CI

Open charting PWA — own the axes, swap the engine

CONTENTS

01 Overview

02 Local Dev

03 Cli

04 Build And Serve

05 Desktop

06 Cloudflare

07 Vps Demo

08 Cors And Origins

09 Ci And Testing

10 Okf

OVERVIEW

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "DevOps" description: "Build, serve, Cloudflare®, VPS demo, CORS, and CI for the AXIS PWA and Worker."

DEVOPS

Abstract

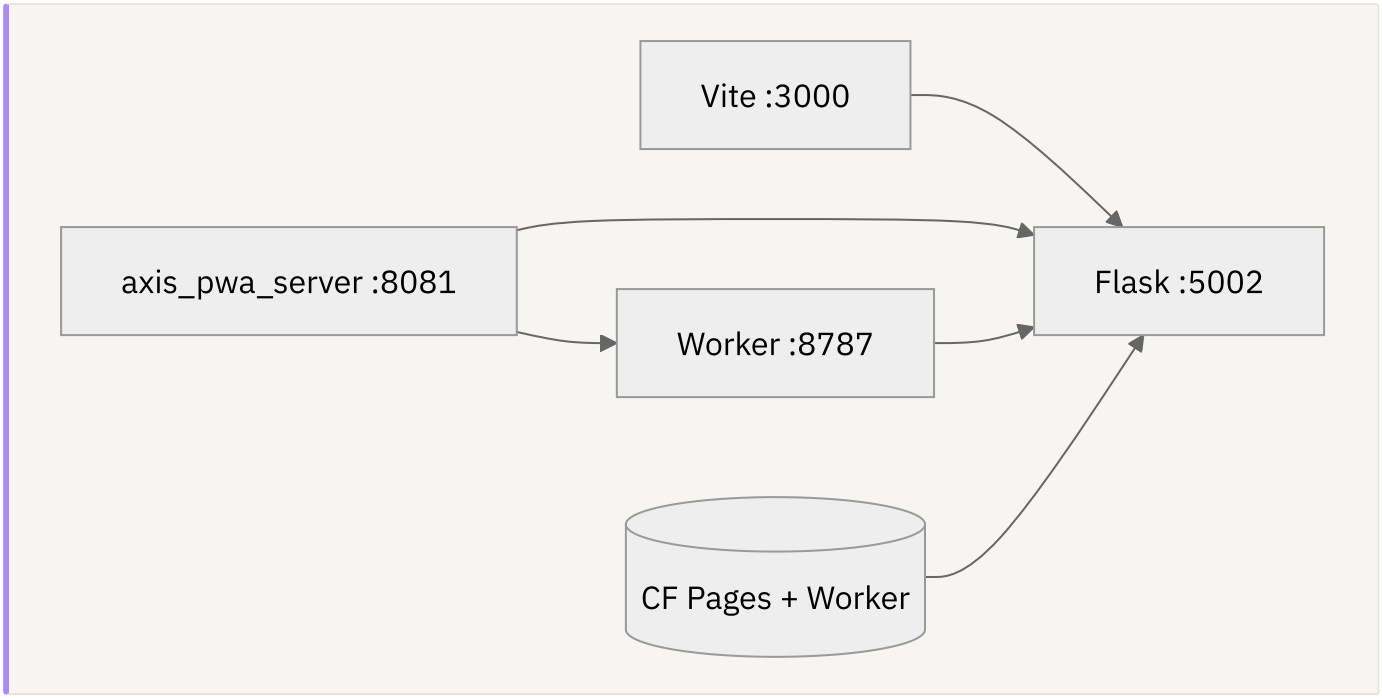
AXIS DevOps spans five runnable surfaces:

1. **Vite + Solid PWA** — product UI (`bun run dev`)
2. **Static server** (`axis_pwa_server.py` or Vite preview) — production-like assets
3. **Cloudflare® Worker** (`worker/`) — edge API / WS / OAuth / on-chain proxy
4. **Desktop (Tauri 2)** — native shell around the same UI (`bun run desktop:dev`)
5. **AXIS CLI** (`@hoox-sh/axis-cli` **0.3.6** under `packages/cli`) — install, doctor, setup, secrets, deploy, health, prewarm, MCP

See Desktop (Tauri) and AXIS CLI for operator entry points.

Product stamp is AXIS **2.27.0**. Python evaluation defaults to the public PYNE host (`https://pynescript.online`); desk workflows still often use a local Flask Pro API (`make run` on `:5002`). The Worker proxies there when `EXTERNAL_BACKEND` is set.

Conceptual model



Topology cheat sheet

Mode	Command sketch	Ports
Dev AXIS	`bun run dev` / `bun run axis dev`	Vite (often `:3000`)
Dev + Flask	+ `make -C ../pyne run` (checkout sometimes named `pynescript`)	`:5002`
Static dist	`bun run build` + `python axis_pwa_server.py`	`:8081`
Worker local	`bun run axis dev worker` / `cd worker && bun run dev`	`:8787`
CF Worker prod	`axis deploy`	HTTPS workers.dev / custom
Operator	`axis doctor` · `axis setup` · `axis health`	—

Page map

Page	Topic
Local dev	Day-to-day loop
AXIS CLI	Install, doctor, setup, secrets, deploy, health, prewarm, MCP (0.3.6)
Build and serve	Vite build, SPA server, assets
Cloudflare®	Pages + Worker deploy
Desktop (Tauri)	Native shell
VPS demo	Single-box static + Flask
CORS and origins	Allowed origins matrix
CI and testing	GitHub Actions, coverage, e2e
OKF bundle	Compiled repo map, refreshed on commit

Frozen names

Surface	Value
Pages project	`axis`
Pages URL	`https://axis.hoox.sh`
Worker script	`worker-axis`
Worker URL	`https://worker.axis.hoox.sh`
Health service	`worker-axis`

See also

- Worker
- Testing reference
- AXIS hub

LOCAL DEV

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

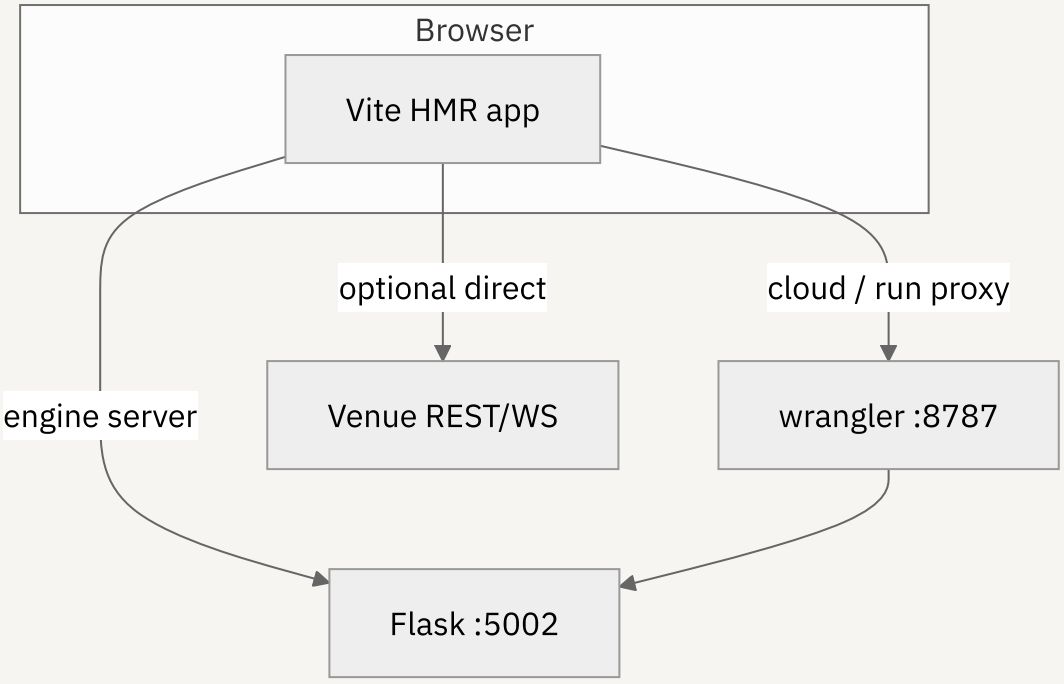
title: "Local development" description: "Day-to-day AXIS loop: Vite, Flask, Worker wrangler, endpoints, and plugin examples."

LOCAL DEVELOPMENT

Abstract

Local AXIS development is a **multi-process** topology. AXIS is a Solid app under Vite; evaluation usually hits Flask; optional cloud/storage/stream features hit the Worker.

Conceptual model



Interface surface — commands

Bootstrap (CLI-first)

```
axis install   # app + worker + CLI
axis doctor    # missing worker/wrangler.toml is a warning until `axis setup`
```

Frontend (preferred product path)

```
bun run dev            # Vite + Solid → typically http://127.0.0.1:3000
# or: bun run axis dev
```

Desktop (optional Tauri shell)

```
bun run desktop:dev    # Vite + native window (see devops/desktop)
# or: bun run axis dev desktop
```

Flask Pro API (engine backend)

From the pyne checkout (sister repo; clone hoox-sh/pyne as `../pyne` — the local directory is sometimes still named `pynescript`):

```
make -C ../pyne run   # Flask → :5002
```

Product default for the **server** engine is `https://pynescript.online` (`src/engines/catalog.ts`). For a local Flask desk, set the PWA Backend URL to `http://127.0.0.1:5002`. Workers Manager and Settings can probe either URL. CLI PYNE helpers (`axis prewarm`, `axis usage`, `keys --pyne`) default via `AXIS_PYNE_URL` → `https://pynescript.online`.

Worker

```
bun run axis dev worker   # http://127.0.0.1:8787
# or: cd worker && bun run dev
```

Point cloud storage / Worker endpoint at `http://127.0.0.1:8787`. For `/api/run` proxy, set Worker `EXTERNAL_BACKEND=http://127.0.0.1:5002` (works on local wrangler; not on Cloudflare® edge).

AXIS CLI

Preferred operator entry for install / doctor / setup / deploy — see AXIS CLI:

```
bun run axis install
bun run axis doctor
bun run axis setup
bun run axis dev worker
```

Makefile helpers

Target	Intent
`python axis_pwa_server.py`	Serve built `dist/` on `:8081` (not Vite)
`make worker-dev`	Wrangler dev
`make worker-install`	Install worker deps
`bun run test:unit`	Bun unit tests under `tests/`

Use `bun run dev` for the product, or `bun run build` and serve `dist/`. The pre-Solid `server.ts` static tree was removed in 2.21.0 (legacy shell).

Storybook (component lab)

```
bun run storybook        # isolated UI on :6006 (`storybook-solidjs-vite`)
bun run build-storybook  # static build
```

Stories live in `src/stories/` (topbar fields, theme tokens, studio primitives, draw tools, toasts, …). Covers chrome in isolation — no Flask/Worker needed.

Debug mode (chart + engine)

Set `VITE_AXIS_DEBUG=1` in `.env` (alias `VITE_DEBUG`) and restart `bun run dev`. The boot banner prints `[axis:debug] chart/engine logging on`. Filter DevTools by `[axis:`.

Prefix	What it logs
`[axis:chart]`	Host mount/unmount, `setData`, live append verdicts (`update` / `setData` / ignored), pan/zoom/menu/drawing actions, `fitContent`
`[axis:engine]`	WebSocket connect/open/close/reconnect, live-wire snapshot/delta/gap/fail, stream apply, apply-transaction abort/commit

Forming ticks and live deltas log at most about once per second so a live feed cannot freeze the console. Snapshots, window slides, menu actions, and apply aborts always log.

Lines are also written to a rotating localStorage ring (`axis.debug.log.0` is current, `.1` – `.3` previous rotations, 64 KiB each). The oldest file is dropped on rotate. Writes coalesce every 250 ms and flush on tab hide. Download the ring as `axis-debug-*.log` from **System Logs** (download icon while debug is on) or `window.__AXIS_DEBUG__.download()` in DevTools. `window.__AXIS_DEBUG__.read()` / `.clear()` / `.stats()` inspect the ring.

Vite inlines the flag at build time. Optional overlay without a rebuild: `localStorage.setItem('axis.debug','1')` then reload. e2e forces the flag off (`playwright.config.ts`).

Internals — useful paths

Path	Role
`src/index.tsx`	App entry
`src/debug.ts`	Debug flag + `[axis:chart]` / `[axis:engine]` logger
`src/debug-log.ts`	Rotating localStorage debug log (`axis.debug.log.*`)
`vite.config.*`	Vite / Solid plugin
`src/engines/catalog.ts`	Default engine endpoint
`worker/wrangler.toml`	Local vars
`public/plugins/`	Example modules served at `/plugins/`

Worked loop

1. Terminal A: `make run` (Flask).
2. Terminal B: `bun run dev`.
3. Open Vite URL; set engine **Server-Side**, Backend URL `http://127.0.0.1:5002` (product default is `https://pynescript.online`).
4. Load `mock-walk` if venues are blocked.
5. Optional Terminal C: Worker with `ALLOW_OPEN_KEYS=1` for cloud library experiments.

Load an example plugin

With Vite or static server:

```
http://127.0.0.1:3000/plugins/example-coingecko-source.js
```

Manager → Plugins → Load from URL.

Invariants & edge cases

1. **CORS** — Flask must allow the Vite origin; Worker `pickOrigin` allows `:3000` and `:8081`.
2. **Pyodide local** — needs vendor wheels under `public/` (copied to dist on build).
3. **Two endpoints** — topbar endpoint for engine vs cloud storage plugin config can differ (Flask vs Worker).
4. `make` prints the Vite URL `:3000`. Compose serves the built PWA on `:8081`.

Failure modes

Issue	Fix
Engine not ready	Flask down / wrong port
CORS errors	Align origins; see CORS
Plugin 404	File only under `src/plugins/` not `public/plugins/`
Worker scripts 401	Set open keys or mint `pn_` key

See also

- Build and serve
- Worker local
- Plugin examples

CLI

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "AXIS CLI" description: "packages/cli — install, doctor, Worker setup (D1/OAuth), secrets, deploy, health — CLI-first operator path."

AXIS CLI

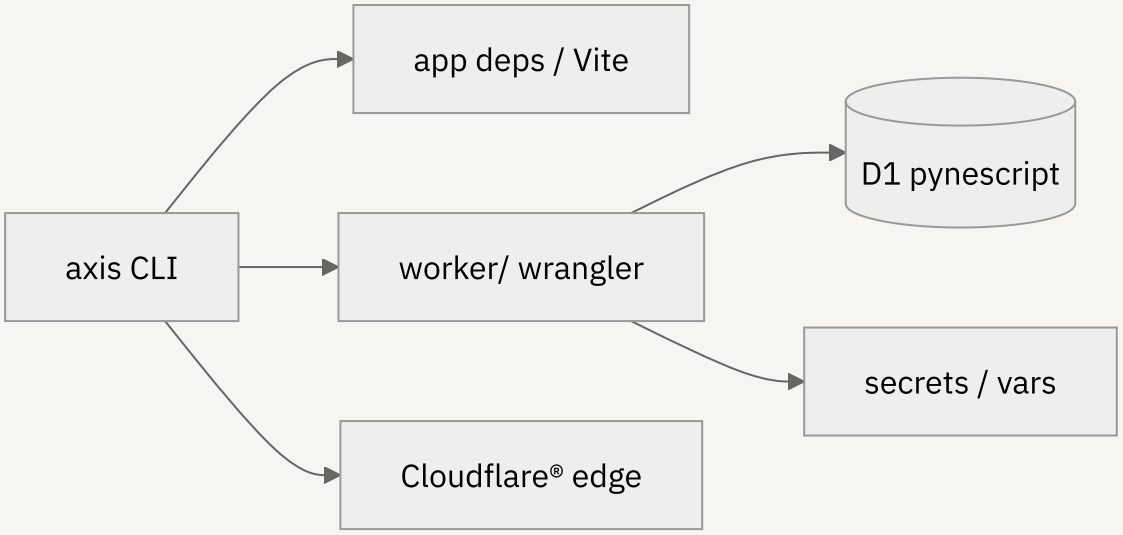
Abstract

The **AXIS CLI** (`@hoox-sh/axis-cli` under `packages/cli`) is the **CLI-first** operator entry for local install, Worker bootstrap, Cloudflare® secrets, deploy, and health checks. It wraps Bun + Wrangler with AXIS-specific paths and defaults (Worker name `worker-axis` , D1 `pynescript` , OAuth device flow).

axis --help listing install, doctor, setup, deploy, health

Requires **Node ≥ 20** to run the published binary (`npm i -g @hoox-sh/axis-cli`) and **Bun ≥ 1.2** for install/dev/build commands that shell out to Bun. Product AXIS is **2.27.0** (`VERSION` / root `package.json`); the CLI package (`@hoox-sh/axis-cli` **0.3.6**) is versioned independently and publishes on `v*` / `cli-v*` tags via `.github/workflows/release.yml` .

Conceptual model



Install

```
# npm
npm install -g @hoox-sh/axis-cli
axis --help

# from monorepo root
bun install
cd packages/cli && bun install && bun run build && cd ../..

bun run axis --help
# Make pass-through
make axis ARGS="--help"
# Optional global link
cd packages/cli && bun link && axis --help
```

Repo aliases (bun scripts & Make targets)

`axis` is the primary interface. These aliases exist for working inside the repo checkout without a global install:

Alias	Runs
`bun run axis -- <args>`	`packages/cli/bin/axis.js`
`bun run axis:install`	`axis install`
`bun run axis:doctor`	`axis doctor`
`bun run axis:setup`	`axis setup`
`bun run axis:deploy`	`axis deploy` (default worker)
`bun run axis:health`	`axis health`
`make axis ARGS="doctor --remote"`	same bin with args
`make axis-install` · `axis-doctor` · `axis-setup` · `axis-deploy` · `axis-health`	fixed wrappers

MCP (agents): `axis mcp --key pn_…` (stdio) · `axis mcp config` — see MCP server.

When using `bun run axis setup -- --flag`, keep the `--` so Bun forwards flags to the CLI.

Command surface

Command	Role
`axis install`	`bun install` root + `worker/` + CLI
`axis doctor` [`--remote`]	Toolchain, CF auth, optional live `/health`. Missing `worker/wrangler.toml` is a warning until `axis setup` (expected on a fresh clone).
`axis setup`	Bootstrap: install → ensure toml → local D1
`axis setup --prod`	Production bootstrap: remote D1 schema + `API_KEYS` KV
`axis setup worker`	Copy `wrangler.toml.example` → `wrangler.toml` if missing
`axis setup d1 --local|--remote` [`--create`]	Apply `worker/schemas/scripts.sql` (scripts + version history)
`axis setup kv [--usage]`	Create and bind `API_KEYS` KV (required when D1 is on)
`axis setup oauth --github-client-id …`	Set public OAuth App id in `[vars]` (or `--secret`)
`axis setup artifacts [--namespace axis-scripts] [--skip-check]`	Bind `SCRIPT_ARTIFACTS` in `wrangler.toml` + verify namespace (git-native script storage, closed beta)
`axis keys create [--tier hobby]`	Mint a `pn_…` key via `POST /api/keys` (`ADMIN_TOKEN`)
`axis keys create --pyne`	Mint on the PYNE host (`POST /auth/create_key`; default `AXIS_PYNE_URL` / `https://pynescript.online`)
`axis keys validate --key pn_…`	Validate a key against the Worker
`axis keys validate --pyne --key …`	Validate against PYNE (`POST /auth/validate`)
`axis secret put|list|delete`	Wrangler secrets (`ADMIN_TOKEN`, `EXTERNAL_BACKEND`, …)
`axis deploy` / `deploy worker`	Apply remote D1 schema, then deploy Worker `worker-axis`
`axis deploy pages`	Vite build + Pages project
`axis deploy all`	Schema + Worker + Pages
`axis health [--oauth] [--scripts] [--url …]`	Probe `/health`, optional OAuth start, optional `/api/scripts`
`axis prewarm [--endpoint …] [--key …]`	Warm Numba builtins / IR cache (`POST /compile/prewarm`)
`axis usage --key pn_…`	PYNE usage (`GET /auth/usage`)
`axis mcp [--key pn_…]`	Stdio MCP proxy to Worker `POST /mcp`
`axis mcp config [--stdio]`	Print MCP client JSON (remote URL or stdio spawn)
`axis whoami`	Cloudflare® account
`axis dev`	Vite product UI
`axis dev worker`	Local wrangler Worker
`axis dev desktop`	Tauri desktop shell

Global flags: `--json`, `--quiet`, `-y` / `--yes`.

Installation self-check

Every `axis` command starts with a lightweight, local-only self-check (skipped for `--version` / `--help`, silenced by `--quiet` / `--json`). It never blocks a command — it only prints stderr warnings when something is off:

- **Runtime compliance** vs `engines` (Node ≥ 20, Bun ≥ 1.2)
- **Install context** — npm install, npx cache, or repo checkout
- **Version drift** — a globally installed CLI older than the repo checkout it is operating on (`npm i -g @hoox-sh/axis-cli` to update)

The same check appears as the optional **CLI installation** row in `axis doctor` (and in `doctor --json` as `cli-install`).

axis doctor: required toolchain and wrangler checks passed

Production checklist

```
axis install
axis doctor
axis setup --prod --github-client-id Ov23li…
axis secret put ADMIN_TOKEN   # comments empty [vars] ADMIN_TOKEN, deploys, then sets secret
axis secret put EXTERNAL_BACKEND
axis deploy all
axis keys create              # --admin-token / AXIS_ADMIN_TOKEN / prompt
axis health --scripts --oauth
axis mcp config --key pn_…    # MCP client snippet (see [MCP server](/axis/docs/worker/mcp))
# Paste Worker URL + pn_… into Settings → Script storage
```

Item	Guidance
`GITHUB_OAUTH_CLIENT_ID`	Public OAuth App id; Device Flow enabled on GitHub; **env wins over body `clientId`**
`GITLAB_OAUTH_CLIENT_ID`	Same for GitLab when using git storage
`ADMIN_TOKEN` / `EXTERNAL_BACKEND`	Prefer `axis secret put`. An empty `[vars]` stub collides with secrets (Cloudflare® 10053)
`API_KEYS` KV	`axis setup kv` then `axis deploy worker`. D1 without KV fails closed (`API_KEYS_REQUIRED`)
`ALLOW_OPEN_KEYS`	`"0"` in production
`REQUIRE_RUN_AUTH`	`"1"` to force Bearer on `/api/run` even without KV
Worker script	`worker-axis` (`https://worker.axis.hoox.sh`)
Pages project	`axis` (`https://axis.hoox.sh`)

Env overrides

Variable	Role
`AXIS_ROOT`	Force monorepo root
`AXIS_WORKER_URL`	Default health / deploy probe URL
`AXIS_ADMIN_TOKEN`	Admin token for `axis keys create`
`AXIS_API_KEY`	Default key for `axis keys validate` / `axis mcp`
`AXIS_PYNE_URL`	Default PYNE Pro API base for `prewarm` / `usage` / `keys --pyne` (fallback **`https://pynescript.online`**)
`AXIS_PYNE_API_KEY`	Default key for `axis prewarm` / `axis usage` / `keys validate --pyne`
`CLOUDFLARE_API_TOKEN`	Non-interactive Wrangler auth
`AXIS_CLI_SRC=1`	Load `src/` instead of `dist/`

Internals

Path	Role
`packages/cli/bin/axis.js`	Bin entry (Bun)
`packages/cli/src/commands/*`	Commander handlers (`install`, `doctor`, `setup`, `keys`, `deploy`, `secrets`, `health`, `prewarm`, `usage`, `dev`, `whoami`, `mcp`)
`packages/cli/src/services/wrangler-toml.ts`	Toml vars, D1 id, KV binding upsert
`packages/cli/src/services/health.ts`	`/health` + OAuth probes
Root `package.json` `axis` / `axis:*` scripts	Convenience wrappers
Root `Makefile` `axis` / `axis-*`	Make wrappers

See also: Installation, MCP (agents), MCP server, Cloudflare® deployment, Worker bindings, Worker auth, packages/cli/README.md.

BUILD AND SERVE

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Build and serve" description: "Vite production build, dist layout, axis_pwa_server.py SPA rules, and asset caching."

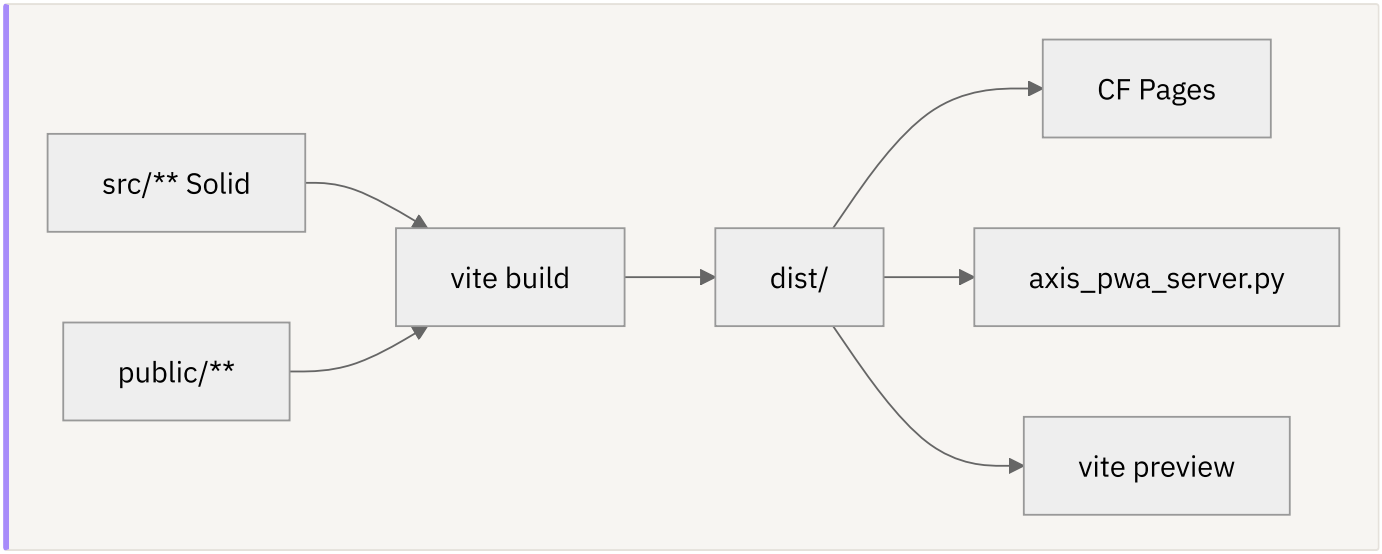
BUILD AND SERVE

Abstract

Production AXIS assets are a **static site** produced by Vite (`bun run build` → `dist`). Serve them with any static host, Cloudflare® Pages, or the repo's `axis_pwa_server.py` SPA-aware server.

Critical requirement: **real binary assets** for Pyodide (`.wasm`, `.whl`, `.zip`) must not be rewritten to `index.html`.

Conceptual model



Interface surface

package.json scripts

```
"dev": "vite",
"build": "vite build",
"preview": "vite preview",
"test": "bun test tests/ worker/tests/",
"test:coverage:gate": "bun run test:coverage && bun scripts/check-coverage.mjs 95",
"test:security": "bun test tests/security/",
"test:e2e:smoke": "playwright test --grep @smoke",
"test:all": "bun run test:coverage:gate && bun run test:security"
```

axis_pwa_server.py

```
bun run build
python axis_pwa_server.py
# HOST / PORT env; default 0.0.0.0:8081 → dist/
```

Behavior (`axis_pwa_server.py`):

Rule	Detail
Root	Serves `dist/`
Cache	`/assets/*` → `immutable` long cache; else `no-cache`
`GET /health` (`/healthz`)	`200` `text/plain` `ok` — never SPA-fallback to `index.html`
`OPTIONS`	`204` with CORS headers (not 501)
`/favicon.ico`	Serves `assets/icon-192.png` when present
SPA fallback	Unknown paths → `/index.html`
Never rewrite	`/health`, `/favicon.ico`, `/assets/`, `/plugins/`, `/vendor/`, `/pyodide/`, or extensions including `.js/.css/.whl/.wasm/.zip/.py/.json…`

This SPA exception list exists because Pyodide + micropip die with opaque `BadZipFile` when HTML is returned for a wheel URL. The engine's `assertZipAsset` detects that class of failure early.

Expected public assets

Path (public → dist)	Purpose
`/plugins/*.js`	Example dynamic plugins
`/vendor/*.whl`	pynescript + antlr wheels
`/pyodide/v0.29.5/**`	Self-hosted Pyodide
`/pyodide/pynescript_runtime.py`	Browser run bridge

Internals

Path	Role
`package.json`	Build scripts
`axis_pwa_server.py`	Threading HTTP SPA server
`src/engines/catalog.ts`	Asset URL expectations
`LEGACY.md`	Product path vs old shell

Makefile note

`make pages-deploy` may historically deploy the wrong tree — **prefer**:

```
bun run build
wrangler pages deploy dist --project-name=axis
```

Build-time flags (Vite)

Env var	Effect
`VITE_APP_VERSION`	Stamps the reported version (CI/Docker); falls back to the repo `VERSION` file
`VITE_MAINTENANCE_MODE`	`1` / `true` / `yes` → the build boots only the maintenance page (`src/ui/MaintenancePage.tsx` : dark screen, flickering HOOX mark, major-release teaser) instead of the chart workspace / editor popout
`VITE_AXIS_DEBUG`	`1` / `true` / `yes` → detailed chart-action and engine logs (`[axis:chart]` / `[axis:engine]` in DevTools) plus a rotating localStorage ring (`axis.debug.log.*` , 4×64 KiB). Alias `VITE_DEBUG` . Forming ticks and live deltas are sampled (~1/s). Download from System Logs or `window.__AXIS_DEBUG__.download()` . Optional overlay: `localStorage.axis.debug=1` then reload

Both are inlined by Vite at build time — set them in `.env` (local/CI overrides) or export them in the build environment, then rebuild. Existing process env vars outrank `.env` files, which is how e2e forces the maintenance gate off (`playwright.config.ts` → `webServer.env`). The flag is documented in `.env.example` .

Worked example — static desk demo

```
bun install
bun run build
PORT=8081 python axis_pwa_server.py
# another terminal
make run   # Flask :5002
```

Open `http://127.0.0.1:8081` , engine endpoint `http://127.0.0.1:5002` .

Invariants & edge cases

1. **Service workers / offline** — product path uses Vite PWA assets; the legacy root `sw.js` was removed in 2.21.0. Only `public/sw.js` ships.
2. **Relative pyodide index** — resolves against `location.origin` .
3. **Do not hand-edit dist** — regenerate from build.

Failure modes

Symptom	Cause
Blank routes on refresh	Server lacks SPA fallback
Pyodide HTML wheel error	Fallback rewriting `.whl` or missing vendor
Missing plugins	`public/plugins` not copied (check Vite publicDir)

See also

- Local dev
- Cloudflare®
- Engines

DESKTOP

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Desktop (Tauri)" description: "Run and package AXIS as a native desktop app with Tauri 2."

DESKTOP (TAURI)

Abstract

AXIS ships as a browser PWA and as an optional **desktop shell** built with Tauri 2. The Solid/Vite UI is unchanged; Tauri embeds it in a system webview (WebKitGTK on Linux, WKWebView on macOS, WebView2 on Windows).

Commands

From the repo root (after `bun install`):

```
bun run desktop:dev     # Vite :3000 + Tauri window (hot reload)
bun run desktop:build   # Vite production build + native installers
bun run desktop:info    # toolchain / webview diagnostics
```

Script	What it does
`desktop:dev`	Runs `tauri dev` → `beforeDevCommand` starts Vite, opens a native window on `http://127.0.0.1:3000`
`desktop:build`	Runs `tauri build` → `bun run build` then packages `dist/`
`desktop:info`	Prints Rust, system libraries, and webview status

Installers land under `src-tauri/target/release/bundle/` (AppImage, deb, rpm, msi, dmg — depending on host OS and `bundle.targets`).

CI (GitHub Actions)

Every **push to `main`** (and PRs that touch desktop/frontend paths) runs `.github/workflows/desktop.yml`:

Trigger	Result
Push `main`	Matrix build: Linux, macOS arm64, macOS x64, Windows → **workflow artifacts** (14-day retention)
Tag `v*` / `desktop-v*`	Same matrix + **GitHub Release** assets attached to the tag
`workflow_dispatch`	Manual re-run

Artifacts are named `axis-desktop-{linux\|macos-arm64\|macos-x64\|windows}-{sha}`. Concurrency cancels superseded runs on the same ref so each push rebuilds cleanly.

Prerequisites

Always

- Bun (or another JS package manager that can run the scripts)
- Rust (`rustc` / `cargo`) — see `src-tauri/Cargo.toml` `rust-version`
- System webview libraries for your platform (Tauri prerequisites)

Linux (Arch / CachyOS example)

```
sudo pacman -S --needed webkit2gtk-4.1 base-devel curl wget file \
  openssl appmenu-gtk-module libappindicator-gtk3 librsvg patchelf
```

Debian/Ubuntu equivalents use `libwebkit2gtk-4.1-dev` and related `-dev` packages (see the Tauri docs for the current list).

Engine backends

Desktop AXIS is the same product as the PWA. For Pine evaluation you still need one of:

Engine	Notes
Local **pyne** Pro API	Clone hoox-sh/pyne as `../pyne` (`make run` → `:5002` ; Vite proxies `/run` in dev). Local checkouts are sometimes still named `pynescript` .
Cloudflare® Worker	Configure worker URL in Manager
Pyodide (in-webview)	Fully offline path; largest first load

Layout

Path	Role
`src-tauri/`	Rust host, `tauri.conf.json` , icons, capabilities
`src-tauri/tauri.conf.json`	App id `sh.hoox.axis` , window size, build hooks
`src/`	Unchanged Solid product UI
`dist/`	Vite output consumed by `frontendDist`

Behaviour notes

- **Service worker** is skipped in the Tauri shell (`src/pwa/register-sw.ts`). Offline install is a PWA concern; the desktop app is already installed as a native binary.
- **Window** defaults: 1440×900, min 960×640, centered.
- **CSP** is left open (`null`) so external venues, Pyodide assets, and worker APIs work the same as in the browser. Tighten later if you ship a locked-down build.

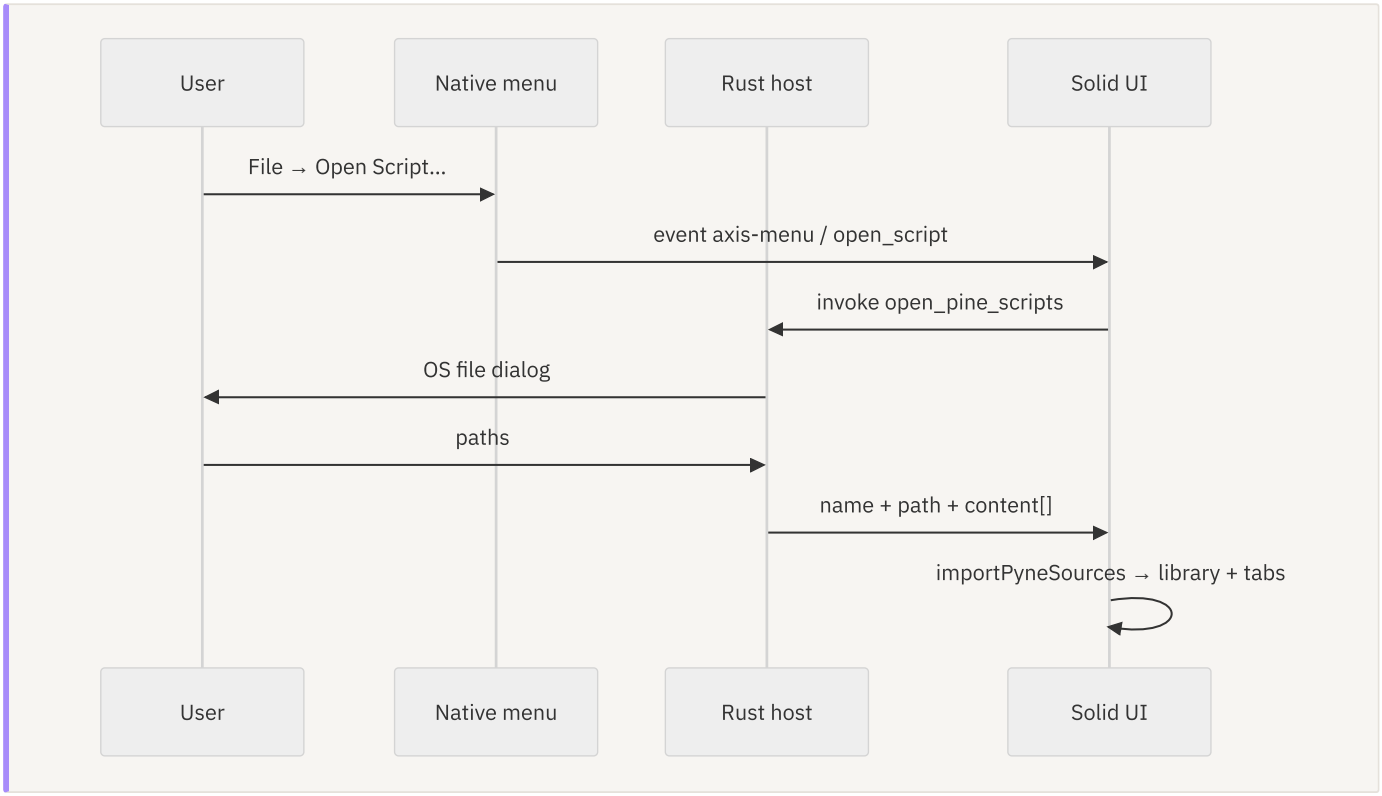
- Icons are generated from `public/assets/icon-512.png` via `bunx tauri icon`.

Native menu & open from disk

The Tauri host builds a native app menu:

Menu	Item	Action
File	**Open Script…** (`⌘/Ctrl+O`)	System multi-file dialog → library + editor tabs
File	**Quit AXIS**	Quit (platform predefined)
Help	**About AXIS**	Info dialog (version / license / site)

Flow

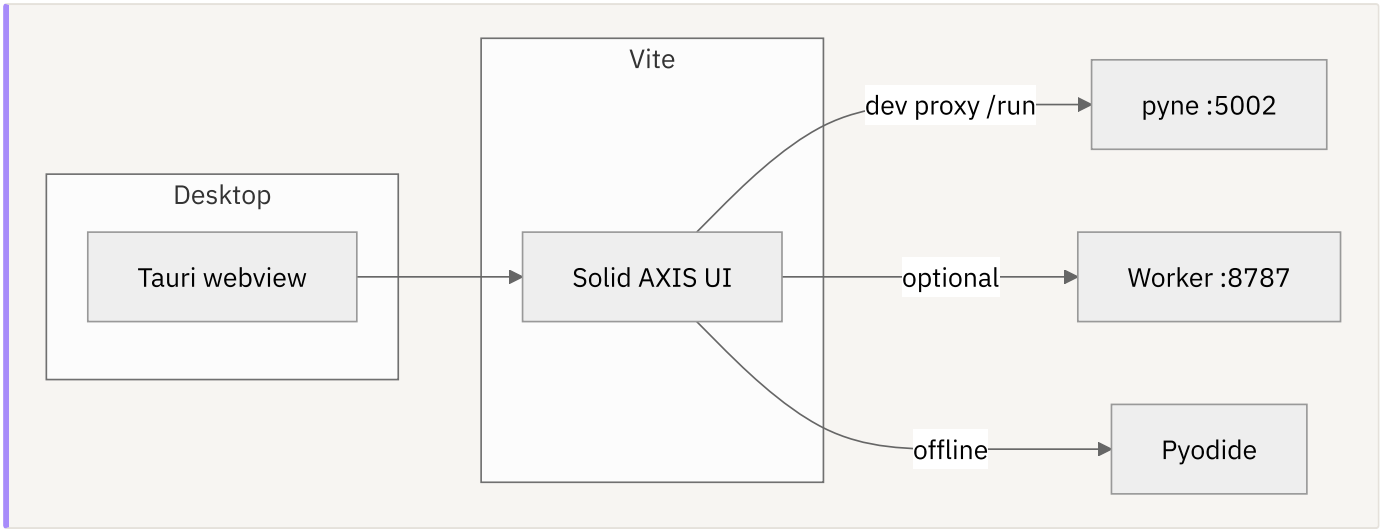


Path	Role
`src-tauri/src/lib.rs`	Menu, `open_pine_scripts` command, dialog + disk read
`src/desktop/`	`isTauriShell` , menu listen, open + About wiring
`src/storage/import-pyne-open.ts`	Shared status/logs/editor open after import
`src/storage/import-pyne-files.ts`	`importPyneSources` (text) + `importPyneFiles` (browser `File`)

Accepted extensions match the PWA drop path: `.pyne` , `.pine` , `.pinescript` , `.pinev5` , `.pinev6` . Per-file size cap on the host: **8 MiB**.

Drag-and-drop of the same extensions still works in the desktop webview.

Conceptual model



CLOUDFLARE

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "Cloudflare® deployment" description: "Pages + Worker for AXIS: Pages axis.hoox.sh (project axis), Worker worker.axis.hoox.sh, CLI deploy, bindings, security checklist."

CLOUDFLARE® DEPLOYMENT

Abstract

Production AXIS on Cloudflare® is a **split deploy**:

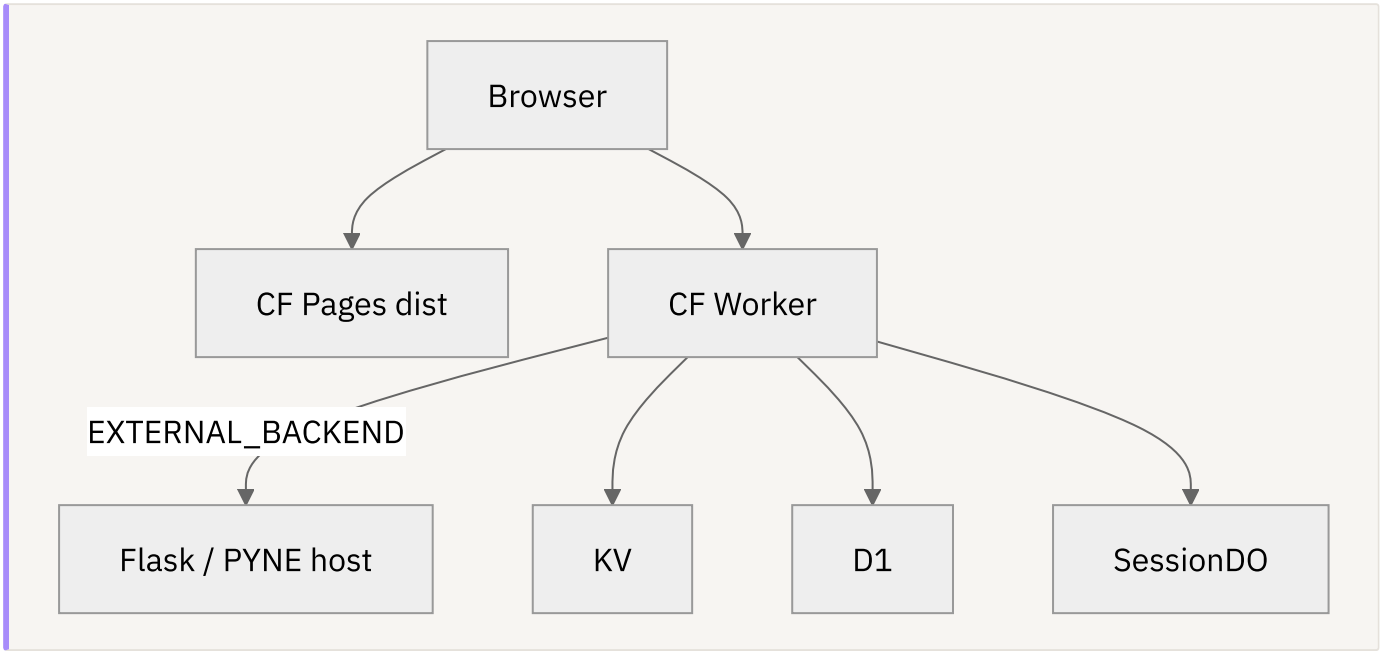
Piece	Role
Pages	Static PWA (`dist/`)
Worker	JSON API + WebSocket (`worker/`)

Pages: project `axis` , canonical `https://axis.hoox.sh` . **Worker:** script `worker-axis` at `https://worker.axis.hoox.sh` .

Evaluation truth: Worker `/api/run` **proxies to** `EXTERNAL_BACKEND` unless the gated Pyodide path is fully implemented and enabled. You still need a Python host (VPS Flask, container, etc.) for real Pine Script™ fidelity today.

Prefer the **AXIS CLI** for setup and deploy (`axis deploy`).

Conceptual model



Deploy procedure

1. Provision bindings (once)

See Worker bindings:

- KV `API_KEYS`, `USAGE`
- D1 `pynescript` + schema
- Optional R2, Durable Objects

2. Configure secrets / vars

Production checklist:

Setting	Production guidance
`EXTERNAL_BACKEND`	Public HTTPS URL of Flask/PYNE API (`axis secret put`)
`ALLOWED_ORIGIN`	Extra exact origins (comma-separated); product hosts + `*.axis.pages.dev` are built-in — CORS
`ADMIN_TOKEN`	Secret, strong
`API_KEYS` KV	**Bind in prod** — D1 without KV fails closed (`API_KEYS_REQUIRED`)
`ALLOW_OPEN_KEYS`	`"0"` or unset
`REQUIRE_RUN_AUTH`	`"1"` optional force Bearer on `/api/run`
`GITHUB_OAUTH_CLIENT_ID`	Env preferred; never rely on body `clientId` alone
`PYODIDE_IN_WORKER`	`"disabled"` until wheel pipeline ready

3. Deploy Worker

```
# Preferred: AXIS CLI
axis deploy
# or: make axis-deploy

# Schema first (remote)
axis setup d1 --remote

# Equivalent raw wrangler
cd worker
bun install
bun run deploy
```

4. Build & deploy Pages

```
bun run axis deploy pages
# or
bun run build
bunx wrangler pages deploy dist --project-name=axis
# Make: make pages-deploy
```

5. Point the PWA

- Engine endpoint: Worker URL (if path-mapped to `/run`) **or** Flask URL directly
- Cloud storage endpoint: Worker origin
- Ensure CORS allows Pages origin

Internals

Path	Role
`worker/wrangler.toml` / `.example`	Worker name + bindings
`worker/README.md`	Ops narrative
`worker/package.json`	deploy scripts
`packages/cli/`	`axis deploy` / `axis health`
Makefile `axis-deploy` / `pages-deploy` / `worker-deploy`	Convenience targets

Health check

```
axis health
# or
curl -sS https://<worker>/health
# service: worker-axis
```

Invariants & edge cases

1. **Pages ≠ Worker host** — CORS required unless same-origin routing via custom domain routes.
2. **Localhost EXTERNAL_BACKEND** is useless from the edge.
3. **DO migrations** apply on deploy when uncommented.
4. **Observability** enabled in wrangler for log tails (`wrangler tail`).

Failure modes

Issue	Fix
503 NO_BACKEND	Set reachable EXTERNAL_BACKEND
CORS blocked	ALLOWED_ORIGIN mismatch
SPA HTML for wheels	Pages must serve `/vendor` and `/pyodide` as static files
Stream NO_DO	Enable SessionDO bindings

See also

- Worker
- CORS
- VPS demo (backend co-host)

VPS DEMO

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "VPS demo topology" description: "Single-box demo: static AXIS dist + Flask Pro API (+ optional reverse proxy) without Cloudflare®."

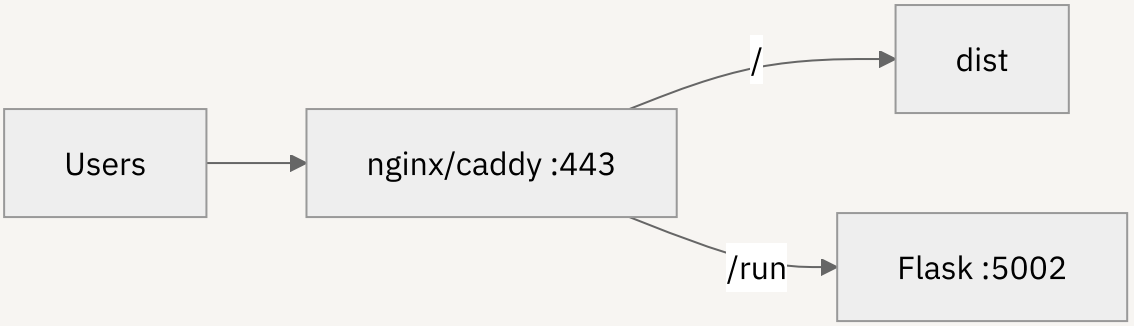
VPS DEMO TOPOLOGY

Abstract

A **VPS demo** is the lowest-surprise production-like setup for Pine evaluation: one machine runs Flask (PYNE) and serves the static AXIS build. Optional nginx/caddy terminates TLS and reverse-proxies `/run` same-origin to avoid CORS.

No Worker required. Cloud storage and DO streams will not work without the Worker.

Conceptual model



Interface surface

Processes

Process	Command	Port
Build (CI or once)	`bun run build`	—
Static	`PORT=8081 python axis_pwa_server.py`	8081
API	`make run` / `python -m backend.app`	5002

Same-origin proxy sketch (Caddy)

```
axis.example.com {
  handle /run* {
    reverse_proxy 127.0.0.1:5002
  }
  handle /optimize* {
    reverse_proxy 127.0.0.1:5002
  }
  handle {
    reverse_proxy 127.0.0.1:8081
  }
}
```

Then PWA endpoint = `https://axis.example.com` and the **server** engine posts to `/run` on the same host.

Production (split: Pages PWA + Hetzner API)

- **PWA** — Cloudflare® Pages custom domain `https://axis.hoox.sh`
- **PYNE Pro API** — Hetzner VPS `https://pynescript.online` (Cloudflare® orange-cloud → origin `204.168.138.51`). nginx `:443` reverse-proxies `/run`, `/optimize`, `/ws/`, `/health`, `/lsp/`, `/datafeed/`, … to gunicorn `127.0.0.1:5002`
- PWA **Backend URL** = `https://pynescript.online` (cross-origin; product CORS regex)
- Cloudflare® Pages previews (`*.axis.pages.dev`) use the same Backend URL — pyne must allow those Origins on `/health` and `/run` (always-on product regex; see CORS)

SSH alias: `ssh pynescript` (`204.168.138.51`). The previous combined VPS (`ssh axis` / `162.254.38.194`) stays until decommission.

Update deploy (from a machine that can SSH to the VPS):

```
# 1. Ship latest main
git push origin main

# 2. On VPS (paths may vary — often /root/axis + rsync dist → PWA WorkingDirectory)
# Pro API (from pynescript checkout)
./scripts/deploy_vps.sh
# or:
ssh pynescript 'cd /root/pynescript && git pull --ff-only && systemctl restart pynescript-api'
# PWA: `bun run build` + wrangler pages deploy (axis.hoox.sh)
```

Local operator CLI for the **Worker** edge (separate from VPS static): `axis deploy` — see AXIS CLI.

Hardened network (UFW) + health checks

Production VPS pattern (verified on `pynescript.online`):

Surface	Bind / allow	Health URL
UFW	`deny incoming` default; **allow 22/tcp, 80/tcp, 443/tcp**	—
nginx	`0.0.0.0:443` TLS (+ `:80` ACME)	`https://pynescript.online/health` → proxy `127.0.0.1:5002/health`
Pro API (gunicorn)	**`127.0.0.1:5002` only**	loopback; **not** public
PWA static	Cloudflare® Pages (`axis.hoox.sh`)	not on this VPS
fail2ban	`sshd` jail	—

Do not open UFW for `5002`, `80`, or `8787` on a public VPS. Direct probes to `http://<vps-ip>:5002/health` **must time out** if hardening is correct.

Workers Manager browser probes:

- On **product API host** (`pynescript.online`), pyne Pro is probed via **same-origin** `/health` (nginx), not `http://127.0.0.1:5002` (that would hit the *client* PC and always look “down”).
- From **CF Pages** (`axis.hoox.sh` / `*.axis.pages.dev`) catalog `publicEndpoint` (`https://pynescript.online`) is used — never client loopback.
- CF Worker health stays on `https://worker.axis.hoox.sh/health` (edge; independent of VPS UFW).

Minimal without TLS

- Static `:8081`, Flask `:5002`
- Prefer a hostname over raw IP when the page is HTTPS
- Open firewall; accept CORS from static origin (configure Flask accordingly)

Internals

Concern	Notes
Pyodide offline	Ship full `dist` including `vendor` + `pyodide`
Process supervisor	systemd units for flask + axis_pwa_server
Updates	Rebuild dist artifact; restart static only
Secrets	Flask admin / pro keys separate from Worker

Worked example — systemd sketch

```
# /etc/systemd/system/axis-pwa.service
[Service]
WorkingDirectory=/opt/axis
Environment=PORT=8081
ExecStart=/usr/bin/python3 axis_pwa_server.py
Restart=on-failure
```

Pair with an existing `backend` unit for Flask.

Invariants & edge cases

1. **CPU/RAM** — Pyodide is browser-side; VPS load is Flask evaluate concurrency.
2. **Do not expose open admin tokens**.
3. **Disk** — dist + wheels are tens of MB.
4. Hybrid: VPS Flask as `EXTERNAL_BACKEND` for a CF Worker still works if VPS has a public URL.

Failure modes

Issue	Fix
Mixed content	HTTPS page calling HTTP Flask blocked
CORS	Prefer same-origin proxy
502 on /run	Flask crashed; check journalctl

See also

- Build and serve
- Cloudflare®
- CORS

CORS AND ORIGINS

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "CORS and origins" description: "Worker pickOrigin: local-dev, product hosts, project-scoped Pages previews, ALLOWED_ORIGIN list, Flask constraints."

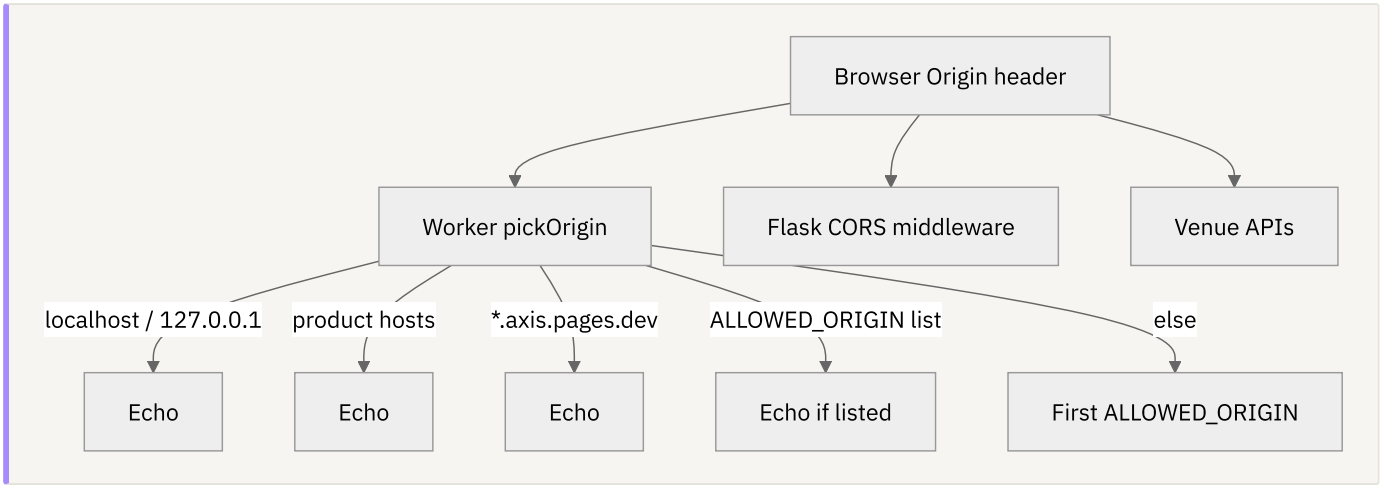
CORS AND ORIGINS

Abstract

AXIS is a browser product that talks to **cross-origin** backends (Flask, Worker, venues). CORS misconfiguration is the #1 local-deploy footgun after missing wheels.

The Worker implements an explicit origin picker (`pickOrigin` in `worker/src/index.ts`); Flask and venue APIs have their own rules.

Conceptual model



Worker behavior

Resolution order for `Access-Control-Allow-Origin`:

1. Local-dev (always echoed)

Any `http://` or `https://` origin on **`localhost`** or **`127.0.0.1`**, optional port (e.g. Vite `:3000`, `axis_pwa_server` `:8081`, Playwright ephemeral ports).

Not allowed (and not needed): `http://0.0.0.0:…` — browsers do not use `0.0.0.0` as a page origin; binding with `HOST=0.0.0.0` only means "listen on all interfaces."

Localhost/127 allowlisting is **safe for CORS**: only pages actually served from those hosts present that `Origin`. A public site cannot forge `Origin: http://localhost:3000` in a real browser.

2. Product hosts (2.0.1+)

Exact product / HOOX / legacy PYNE hosts under:

- `*.hoox.sh` / `hoox.sh`
- `*.pynescript.online` / `pynescript.online`

Examples: `https://axis.hoox.sh` (canonical Pages), `https://hoox.sh`, `https://pynescript.online`.

3. Project-scoped Cloudflare® Pages (not open `*.pages.dev`)

Canonical Pages is **`https://axis.hoox.sh`** (product host above). Preview hashes on **`*.axis.pages.dev`** (and the apex `axis.pages.dev`) are also echoed. Legacy **`*.pynescript-axis.pages.dev`** still matches. Arbitrary third-party Pages projects (`evil.pages.dev`, other apps) fall through to the allowlist fallback — they are **not** reflected.

4. `ALLOWED_ORIGIN` allowlist

Comma-separated exact origins in `env.ALLOWED_ORIGIN` (e.g. `https://a.example,https://b.example`). If the request Origin is listed, echo it; otherwise return the **first** entry (default `https://pynescript.online`).

CORS headers applied:

```
Access-Control-Allow-Origin: <picked>
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type, Authorization, X-Admin-Token, If-Match
Access-Control-Max-Age: 86400
Vary: Origin
```

`OPTIONS` → `204` preflight.

Production implication

For a custom domain already under product regex, no extra var is required. For one-off preview hosts outside the project, list them:

```
ALLOWED_ORIGIN = "https://my-preview.example.com,https://axis.example.com"
```

Flask / server engine

The **server** engine posts from the browser to the configured endpoint. Flask Pro API must:

1. Answer `OPTIONS` preflight if cross-origin.
2. Reflect or allow the PWA origin.
3. Accept `Content-Type: application/json`.

Pyne always appends the same product Origin regex as Worker `pickOrigin` (including `*.axis.pages.dev`) even when systemd `ALLOWED_ORIGINS` is a short list. `GET /health` (AXIS Settings probe) and `POST /run` are **free CORS paths** — they echo any `Origin` so a Pages preview can reach `https://axis.hoox.sh` without a per-preview allowlist entry.

Same-origin reverse proxy (VPS demo) eliminates CORS when the PWA and Pro API share `https://axis.hoox.sh`. Cross-origin Pages → VPS still needs those headers (they are now default).

Venue sources/streams

Public Binance/OKX/etc. REST must send CORS headers usable by browsers. Firefox often reports **"CORS request did not succeed / status (null)"** when the TCP/TLS connection fails entirely (geo block, firewall, extension) — that is **not** a missing `Access-Control-Allow-Origin` header.

AXIS resilience for Binance:

1. Direct hosts: `api.binance.com` then `data-api.binance.vision` (`src/data/binance-http.ts`).
2. Worker allowlisted proxy: `GET /api/market/binance/klines|ticker/24hr|exchangeInfo` on the AXIS Worker (`worker/src/market.ts`).
3. Live WS rotates `:9443` → `:443` → `data-stream.binance.vision` (port 9443 is frequently firewalled).
4. Offline last resort: `mock-walk` / `mock-poll`, or warm bars-cache.

Prefer the DO relay (`cf-do` stream / `/api/stream`) when many tabs share one upstream WS — still requires the Worker edge to reach Binance.

AXIS resilience for MEXC:

1. Worker allowlisted proxy first: `GET /api/market/mexc/{klines,ticker/24hr,exchangeInfo}` (`src/data/mexc-http.ts`). Public `api.mexc.com` does **not** send `Access-Control-Allow-Origin`.

2. Direct `api.mexc.com` only as last-ditch fallback (offline lab, tests, or Worker 5xx). Worker 4xx is not retried against the CORS-blocked host.

Dynamic plugins

- **Module import** of plugin JS: needs CORS on the **script origin**. Prefer same-origin `/plugins/*` (PYNE Agent ships as `/plugins/axis-pine-agent.js`).
- **Remote** modules (e.g. legacy `pyne-agent-worker…/plugin/axis-pine-agent.js`) need ACAO + CSP `script-src` allow when a CSP is present.
- **fetch inside plugin**: subject to third-party CORS independently (agent API still uses the pyne-agent Worker origin).

Invariants & edge cases

1. **Credentials** — Worker CORS does not set `Allow-Credentials: true`; use Bearer headers, not cookies.
2. **Multiple prod domains** — product regex + comma-separated `ALLOWED_ORIGIN` cover multi-host; unknown sites never get open reflection.
3. **Vite vs 8081** — both covered for Worker; Flask config must match whichever you use.
4. **Health checks** from curl omit Origin — still work; browser calls need correct ACAO.
5. **No open `*.pages.dev`** — only the Pages project `axis` previews (`*.axis.pages.dev`) are product-scoped.

Failure modes

Browser console	Meaning
blocked by CORS policy	Origin not allowlisted
preflight 404	Server lacks OPTIONS
No ACAO on error body	Some error paths forgot headers (Worker try/catch mostly covered)

See also

- Worker bindings
- Local dev
- Dynamic loader

CI AND TESTING

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

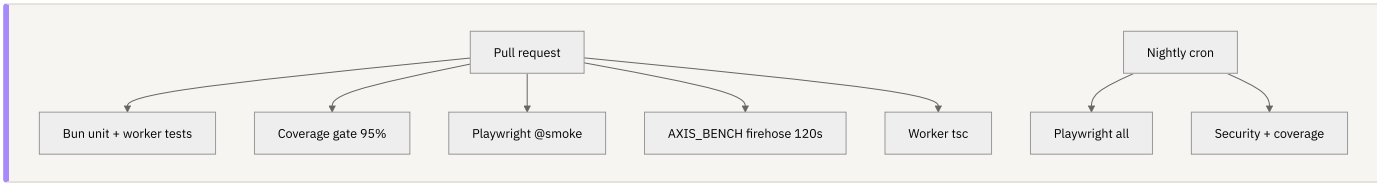
title: "CI and testing" description: "GitHub Actions for AXIS: unit coverage gate, Playwright smoke, worker typecheck, nightly full e2e."

CI AND TESTING

Abstract

AXIS quality gates live primarily under the repo root (Bun tests) and `worker/tests/`, orchestrated by GitHub Actions (`.github/workflows/ci.yml`, `axis-nightly.yml`). Deep test conventions are documented in Testing reference and `TESTING.md`.

Conceptual model



Interface surface — local commands

```
bun run test                # tests/ + worker/tests/
bun run test:unit
bun run test:coverage:gate  # lcov + scripts/check-coverage.mjs 95
bun run test:security
bun run test:e2e:smoke
bun run test:e2e:critical
bun run test:e2e
bun run test:all            # gate + security
bun run test:bench          # AXIS_BENCH=1 firehose (120s wall budget)
bun run perf:audit          # typecheck + hot-path unit + benches → reports/perf-audit.json
bun run perf:audit:full     # also full unit + Playwright @smoke
```

Worker:

```
cd worker
npm run typecheck
```

Repo Makefile:

```
bun run test:unit
make worker-typecheck
```

CI workflows

ci.yml (excerpted jobs)

Job (names vary)	What
Frontend tests / coverage	Bun install, coverage, artifact `axis-coverage-lcov`
`axis-e2e`	Playwright `@smoke` on PR
Firehose budget	`bun run test:bench` (`AXIS_BENCH=1`). Fails if 500k `appendBar` ticks on ~50k history exceed `FIREHOSE_SOFT_BUDGET_MS` (120 s)
Worker typecheck	`worker` tsc
Related	Python suite remains for pynescript core

axis-nightly.yml

Job	What
Playwright full	all e2e specs
Security + coverage gate	`test:coverage:gate` + `test:security`

Schedule: `0 4 * * *` UTC + `workflow_dispatch`.

Internals

Path	Role
`TESTING.md`	Authoritative testing guide
`scripts/check-coverage.mjs`	Scoped line gate
`scripts/perf-audit.mjs`	`bun run perf:audit` — typecheck, hot-path unit, AXIS_BENCH, JSON report
`playwright.config.ts`	webServer build+preview `:4173`
`e2e/`	Specs with `@smoke` / `@critical` tags
`worker/tests/`	auth, keys, scripts, runtime, http, proxy-router, rate-limit
`.github/workflows/ci.yml`	PR gates
`.github/workflows/axis-nightly.yml`	Nightly

Coverage policy (summary)

- **Gate:** 95% lines on a **scoped** core (plugins, storage minus idb, store, results, sources, streams, data, selected chart helpers, worker auth/keys/runtime/scripts).
- **Excluded from gate:** heavy chart UI, pyodide boot, legacy JS, Solid `.tsx` surfaces (unit-tested elsewhere or deferred).

E2E design

- Builds production preview; baseURL `http://127.0.0.1:4173`.
- Mocks `/run` and Binance where possible so smoke does not need live Pro API.
- Selectors use `data-testid` (`axis-btn-load`, `axis-manager`, …).

Storybook

`bun run storybook` (`:6006`) serves `src/stories/` in isolation — visual review without Flask/Worker. Not a CI gate today.

Invariants & edge cases

1. **No real network in unit tests** — mock `fetch` / WebSocket.
2. **Import `./setup` first** when touching store/plugins.
3. **CI Bun version** pinned in workflows (e.g. 1.3).
4. **Playwright browsers** installed with `--with-deps` on CI.

Failure modes

CI red	Likely
Coverage gate	New untested code in scoped paths
Smoke timeout	Build slow; webServer 180s
Worker tsc	Env typing / DO exports
Flaky e2e	Missing testid; race without mock

See also

- Testing reference
- Local dev

OKF

COPYRIGHT (C) 2026 HOOX · AXIS · HOOX-SH (JANGO_BLOCKCHAINED)
THIS FILE IS PART OF AXIS.
AXIS IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
AXIS IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH AXIS. IF NOT, SEE HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-ONLY

title: "OKF bundle" description: "Compiled Open Knowledge Format cache of the AXIS repo, refreshed on every commit."

OKF BUNDLE

Abstract

`okf/` is an Open Knowledge Format v0.2 bundle. It is the compiled map of this repo: one concept per source directory and per doc, with imports already linked. Agents read `okf/index.md` and then one concept instead of searching the tree.

Refresh

The pre-commit hook (`.githooks/` , installed by `bun install`) runs the pipeline against the git index and stages the generated bundle into the commit. Unstaged edits, including unstaged edits to curated playbooks, are not part of that commit's bundle.

```
bun run okf:enrich    # draft, link, and lint from the working tree
bun run okf:check     # fail if the committed bundle would change
bun run okf:lint      # conformance errors and producer warnings
bun run okf:query chart drawings
bun run okf:context src/chart
```

A directory path resolves to that concept. A file path resolves to the deepest directory concept that contains it. Query terms match path segments and the file names listed on a concept, so `ui` does not match `builtins`, and `drawings.ts` finds the directory that contains it. `AGENTS.md` is gitignored and is not part of this bundle. The shared reading guide is `okf/playbooks/read-this-first.md`.

`OKF_SKIP=1` skips the hook refresh. CI runs `okf:check`, so a commit that bypasses the hook still cannot merge a stale bundle.

Generated concepts (`okf_lock: generated`) are replaced when their sources change. Curated playbooks (`okf_lock: human`) are kept. Put hand-written knowledge in `okf/playbooks/` with that lock. The draft records exports, imports, and the first non-license comment. It does not call a model.

Layout

Path	Role
`okf/index.md`	Bundle root. Declares `okf_version: "0.2"`.
`okf/log.md`	Newest-first compile history.
`okf/code/`	One concept per source directory.
`okf/docs/`	Outlines of `docs/`.
`okf/playbooks/`	Curated reading guide and product map.
`scripts/okf/`	Compiler, linter, and query CLI.